Hub RAG Architecture & Deployment Guide

1. Overview: What Has Been Added?

We have evolved the system from a **Single-Site RAG** (standalone) to a **Federated, Multi-Site Architecture** (networked). The new "Hub" layer acts as a central orchestrator that intelligently routes questions to confidential computing nodes without ever storing patient data itself.

New Components Added

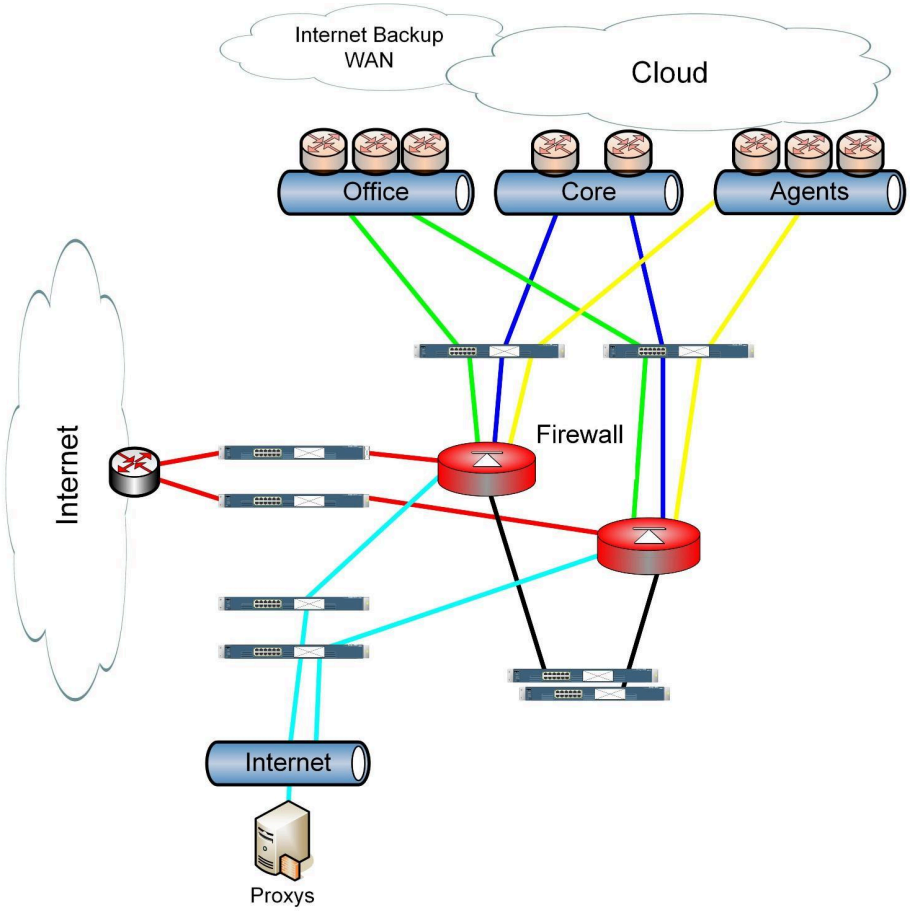
1. **Hub Server (hub_server.py):** The single API entry point for all users. It handles authentication, token generation, and orchestration.
2. **Semantic Vector Router (hub_metadata_db.py):** A specialized Vector Database running on port 9001. It stores "Site Capabilities" rather than patient records. It uses embeddings to match user intent (e.g., "blood test") to specific sites (e.g., "Site 1 - Labs").
3. **Site Manager (site_manager.py):** An asynchronous client that manages parallel connections to multiple distributed sites.
4. **Policy Token Issuer (token_issuer.py):** Generates cryptographic attestations (tokens) that prove to the remote sites that a query is authorized.
5. **Insight Aggregator (aggregator.py):** An LLM-based engine that fuses partial answers from different sites into a single, coherent medical summary.

2. Why Is This Necessary?

The Problems Solved

- **Scalability:** In a real healthcare network with dozens of clinics, a user (doctor/researcher) cannot be expected to know which specific IP address holds a specific patient's data.
- **Data Fragmentation:** Medical data is naturally siloed. Site 1 might have the Pathology report, while Site 2 has the Radiology scan. Previous to this, a user would get two separate, disconnected answers.
- **Security & Attack Surface:** By introducing the Hub, we can hide the IP addresses of the secure Site Enclaves. The Sites can be configured to **only** accept traffic from the Hub, making them invisible to the public internet.

3. How It Works (The Technical Flow)



Shutterstock

Step 1: The User Query

A user sends a prompt to the Hub: *"Give me a summary of Patient PAT001's kidney health."*

Step 2: Semantic Routing (The Brain)

- The **Hub Router** converts the query into a vector embedding.
- It searches its Metadata DB (Port 9001) for the most relevant sites.
- *Result:* It identifies **Site 1** (matches "kidney/creatinine") and **Site 2** (matches "diagnosis"). It ignores Site 3.

Step 3: Secure Dispatch

- The Hub generates a **Policy Token** containing the User's ID, Role (e.g., "Staff"), and Privacy Level (e.g., "Low").
- It sends the query + token to Site 1 and Site 2 simultaneously.

Step 4: Confidential Execution (The Sites)

- **Site 1** receives the request, validates the token, queries its local secure Vector DB,

retrieves "Creatinine 1.4", and scrubs PHI.
- **Site 2** receives the request, validates the token, retrieves "CKD Stage 2", and scrubs PHI.

Step 5: Aggregation (Data Fusion)

- The Hub receives the two distinct answers.
- The **Aggregator** uses an LLM to synthesize them: *"The patient has elevated creatinine (1.4) indicating renal insufficiency, consistent with their diagnosis of Chronic Kidney Disease Stage 2."*

4. How to Test (Local Simulation)

To test the full system locally, we simulate the distributed network using different ports on localhost. You need **8 Terminals** active.

A. Infrastructure Setup

1. **Terminal 1 (Patient Data Store):**
   - cd Confidential-Computing-RAG
   - python mock_vector_db.py (Runs on Port 8000)
2. **Terminal 2 (Site 1 - Labs):**
   - export SITE_ID="site_1" && uvicorn api_gateway:app --port 8080
3. **Terminal 3 (Site 2 - Diagnosis):**
   - export SITE_ID="site_2" && uvicorn api_gateway:app --port 8081
4. **Terminal 4 (Site 3 - History):**
   - export SITE_ID="site_3" && uvicorn api_gateway:app --port 8082
5. **Terminal 5 (Router Data Store):**
   - cd Confidential-Computing-Hub
   - python hub_metadata_db.py (Runs on Port 9001)

B. Data Injection (Run Once)

6. **Terminal 6 (Seeder):**
   - cd Confidential-Computing-Hub
   - python populate_router.py (Teaches the Hub where sites are)
   - cd ../Confidential-Computing-RAG
   - python populate_sites.py (Gives the Sites actual patient data)

C. The Hub

7. **Terminal 7 (Orchestrator):**
   - cd Confidential-Computing-Hub
   - uvicorn hub_server:app --port 9000

D. Run a Test Query

8. **Terminal 8 (User):**

```
curl -X POST http://localhost:9000/hub/query \
-H "Content-Type: application/json" \
-d '{
   "query": "Give me a full summary of patient PAT001",
   "user_id": "Dr_Smith",
   "role": "pi",
   "privacy_level": "none"
}'
```

5. Deployment Guide: Translating to Remote EC2

Moving from localhost to a real AWS deployment involves mapping ports to **IP Addresses** and configuring **Security Groups**.

Step 1: Server Provisioning

Instead of terminals, you will provision **4 separate EC2 instances** (or AWS Nitro Enclaves).

Server Role	Software Running	Port	Internal IP (Example)
Site 1	api_gateway.py	8080	10.0.1.10
Site 2	api_gateway.py	8080	10.0.1.11
Site 3	api_gateway.py	8080	10.0.1.12
Hub	hub_server.py, hub_metadata_db.py	9000, 9001	10.0.1.5

Step 2: Update Hub Routing Table

On the **Hub Server**, you must update the routing data to point to the real internal IP addresses of the sites, not localhost.

Modify populate_router.py on the Hub:

```
SITES = [
  {
    "text": "SPECIALTY: Laboratory...",
    "metadata": {"site_id": "site_1", "url": "[http://10.0.1.10:8080](http://10.0.1.10:8080)"}
  },
```

```
  {
    "text": "SPECIALTY: Diagnosis...",
    "metadata": {"site_id": "site_2", "url": "[http://10.0.1.11:8080](http://10.0.1.11:8080)"}
  },
  # ... etc
]
```

Run python populate_router.py on the Hub instance to update the Vector DB.

Step 3: Configure Security Groups (Firewall)

This is critical for the "Confidential" aspect.

1. **Site Security Groups (The Enclaves):**
    - **Inbound Rules:** Allow TCP Port 8080.
    - **Source:** Restrict strictly to the **Hub's Private IP** (10.0.1.5/32).
    - *Effect:* No one on the internet can query the hospitals directly. They *must* go through the Hub.
2. **Hub Security Group:**
    - **Inbound Rules:** Allow TCP Port 9000 (API access) from your VPN or authorized client IPs.
    - **Outbound Rules:** Allow traffic to the Site IPs on Port 8080.

Step 4: Environment Variables

Ensure every EC2 instance has a .env file containing:

- OPENAI_API_KEY (Required for RAG generation)
- SITE_ID (Specific to each site instance)